COMPUTACION 2021

Clase 2

Temas de la clase 2

- ✓ Etapas de la resolución de un problema utilizando una computadora
- ✓ Razonamiento top-down
- ✓ Otros Tipos de Datos
- ✓ Expresiones

Etapas en la resolución de problemas utilizando una computadora

- Análisis del problema
- Diseño del algoritmo
- Codificación
- Compilación y ejecución
- Verificación y depuración
- Documentación

ERRORES en Programación

- Los **errores sintácticos** son los que se cometen cuando no se siguen exactamente las reglas de escritura de un lenguaje de programación.

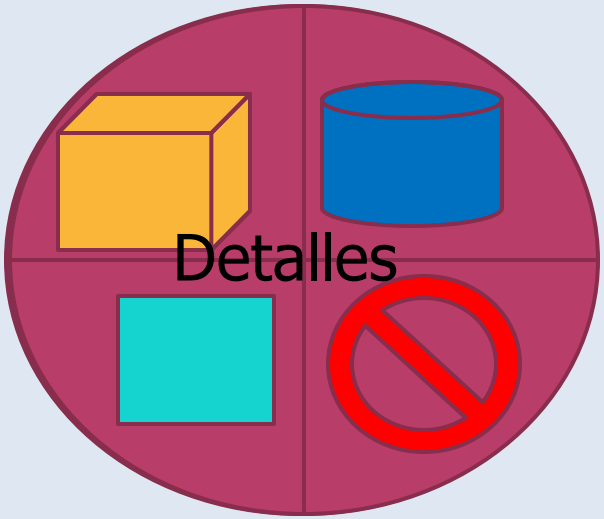
  (SINTAXIS: define como se combinan los símbolos, constantes, identificadores y palabras reservadas)

- Los **errores semánticos** son los que se cometen cuando los resultados obtenidos no son los esperados como solución al problema planteado.

Diseño del algoritmo Razonamiento top-down

(deducción de arriba hacia abajo)

En el modelo de razonamiento **Top-down** se formula, en principio, un resumen del algoritmo, sin especificar detalles.



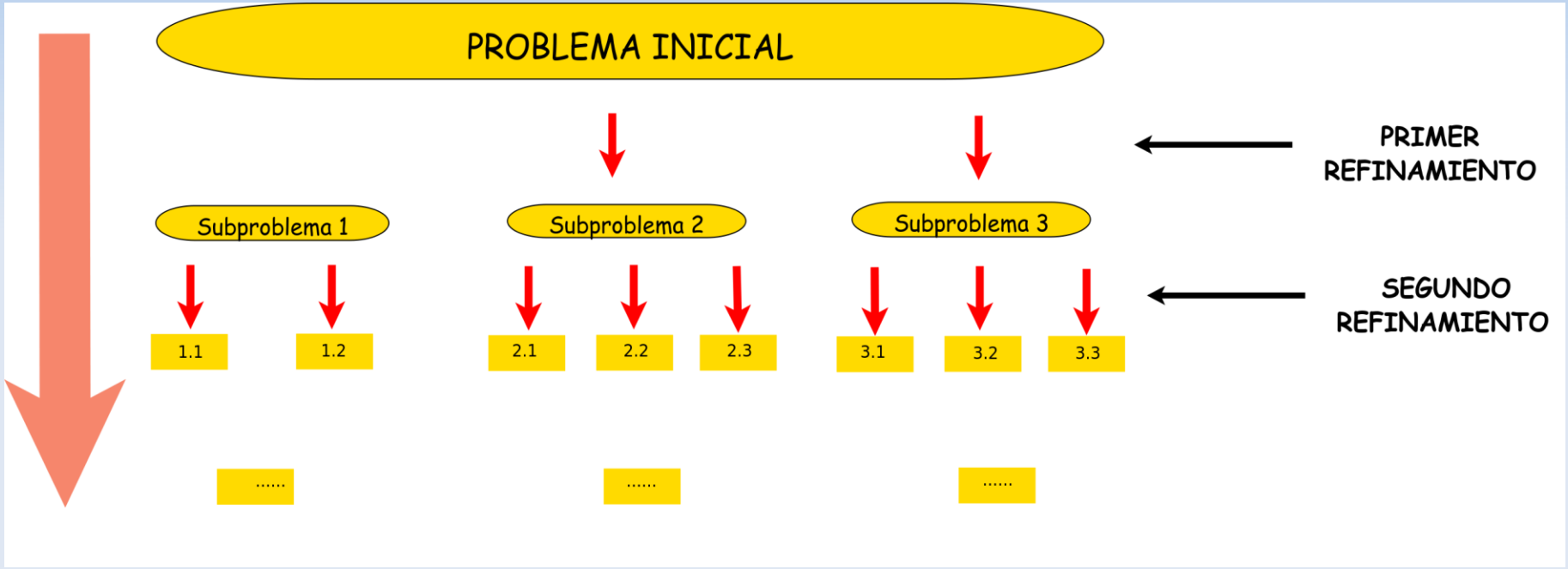
Razonamiento top-down

(deducción de arriba hacia abajo)

Cada parte del programa se refina diseñando con mayor detalle. Esto implica una redefinición mas precisa, hasta que la especificación completa es lo suficientemente detallada para codificar el programa.

Se usa "cajas negras" que hacen más fácil cumplir requerimientos aunque estas cajas negras no expliquen en detalle los componentes individuales.

Razonamiento top-down (deducción de arriba hacia abajo)



Ejemplo

Calcular el seno usando un polinomio de Taylor

$$seno(x) \simeq p_7(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!}$$

<u>Acciones primitivas del procesador</u>:

ingresar datos, escribir datos,

sumar, restar, multiplicar y dividir

ALGORITMO SENO1

1. Leer x
2. Calcular $p_7(x)$
3. Escribir el resultado

Acción no primitiva

Acciones no primitivas derivadas

ALGORITMO SENO2

1. Leer x

2.1 Calcular los términos de la serie

2.2 Sumar algebraicamente los términos de la serie

3. Escribir el resultado

ALGORITMO SENO3

1. Leer x

2.1.1 $a \leftarrow x^3/3!$

2.1.2 $b \leftarrow x^5/5!$

2.1.3 $c \leftarrow x^7/7!$

2.2 sen $\leftarrow$ x - a + b-c

3 Escribir sen

Acciones no primitivas derivadas

ALGORITMO SENO4

1. Leer x

2.1.1.1 pot1 ← x*x*x

2.1.1.2 fact1 ← 3*2

2.1.1.3 a ← pot1/fact1

2.1.2.1 pot2 ← x*x*pot1

2.1.2.2 fact2 ← 5*4*fact1

2.1.2.3 b←pot2/fact2

2.1.3.1 pot3 ←x*x*pot2

2.1.3.2 fact3 ←7*6*fact2

2.1.3.3 c←pot3/fact3

2.2 sen ← x - a + b-c

3 Escribir sen

Las acciones encontradas son primitivas.

Puede fácilmente llevarse a un programa Pascal

Uso de la sección TYPE: datos creados por el programador

- Para que el programa incorpore un nuevo tipo de datos le debemos dar un **nombre** en una declaración de tipo.

- La declaración del nuevo tipo debe hacerse en la sección TYPE.

- Notar que esta declaración es **distinta** a la declaración de variables.

- Una vez que se declaró el nuevo tipo de dato, se pueden declarar **variables,** de dicho tipo, en la sección VAR

PROGRAM Identificador
(**USES** Identificador)

CONST

Definición de constantes

TYPE

Declaración de tipos

VAR

Declaración de variables

PROCEDURE

Declaración de procedimientos

FUNCTION

Declaración de funciones

BEGIN

Sentencias ejecutables

END.

Vamos a analizar la sección de declaración de tipos de datos

Uso de la sección TYPE: datos creados por el programador

Tipos de datos ordinales

- **No son tipo de datos, sino una clasificación.**
- **Se clasifican como ordinales a aquellos tipos de datos cuyos valores poseen un orden determinado (predefinidos o definidos por el programador)**

1. Tipos enteros (byte, word, integer)
2. Tipo char
3. Tipo boolean
4. Tipo subrango

Datos subrango

- **El tipo de dato SUBRANGO es un subconjunto de un tipo asociado (que comprenden a los tipos ordinales: INTEGER, CHAR o incluso BOOLEAN)**

- **Se define como un subintervalo de algún otro tipo escalar u ordinal.**

- **La definición indica el valor menor y mayor en el subrango (la cota inferior no puede ser mayor que la cota superior)**

- ***No se permiten*** **subrangos del tipo real ni string.**

Datos tipo su

Estas constantes deben ser distintas y del mismo tipo

Sintaxis:

TYPE
Identificador de tipo=constante1..constante2;

```
TYPE
Negativos=-1000..-1;{subrango de integer}
Siglo_XXI=2001..3000;{subrango de integer}
Minusculas='a'..'z'; {subrango de char}

VAR
D1,D2:Siglo_XXI;
letra:Minusculas;
Num1,num2:negativos;
```

Extendiendo el poder de constantes y variables: EXPRESIONES

- La combinación de valores constantes y variables relacionadod mediante ***operadores*** generan expresiones

- La característica fundamental de una expresión es la obtención de ***un valor*** luego de su cálculo.

Formato de las expresiones

Expresión binaria

operando *operador* operando

Ejemplo: A + B

Expresión unaria

operador operando

Ejemplo: - A

TIPOS DE EXPRESIONES

- ARITMETICAS
- RELACIONALES
- LOGICAS
- CARACTER

EJEMPLO

A:=34.6*5/(D+25); Exp ARITMETICA y Sentencia de asignacion

Operandos de expresiones aritméticas

* multiplicación

/ división real {ambos operandos pueden ser enteros, pero el resultado es siempre real}

div cociente entero {divide y trunca, solo para enteros}

mod resto entero {a mod b=a-[(a div b)*b], sólo para enteros}

\+ suma

\- resta

Expresiones aritméticas ejemplos

7 div 2

12 / 6.2

7 div 2 + 12 / 6.2

7 div 2 + 12 / 6.2 - A*2.1

beta * alfa / 16.87654

El resultado de todas las expresiones es un único valor.

Expresiones relacionales

- Las **expresiones relacionales** se utilizan para comparar expresiones, obteniéndose como resultado un valor booleano (true o false).

- Los operadores relacionales sirven para comparar dos valores por mayor, menor, igual , mayor o igual, menor o igual, o distinto.

Operadores relacionales

=	igual
<>	distinto
>	mayor
<	menor
<=	menor o igual
>=	mayor o igual

4.5>22.3	resultará siempre ***false***
2.2=2.2	resultará siempre ***true***
A<=2.3	el resultado depende del valor de A
w+7.8 = a/b-9	el resultado depende de los valores de **w, a** y **b.**

Expresiones lógicas

- Las expresiones relacionales (con resultados booleanos: **true** o **false**) pueden combinarse mediante los operadores lógicos **and** (conjunción), **or** (disyunción) y **not** (negación).

- Esta combinación origina las **expresiones lógicas** (o expresiones booleanas) que amplían las posibilidades para modelar predicados (o condiciones).

- Estos predicados serán evaluados para controlar de flujo de ejecución del programa.

- La importancia de las expresiones lógicas se verá con las sentencias ejecutables IF, WHILE y REPEAT.

Expresiones lógicas
Tablas de verdad

Si p y q son operandos lógicos (expresiones relacionales) se obtienen las siguientes tablas de verdad:

p	q	p and q	p or q	not p
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

Expresiones lógicas reglas de precedencia

En toda expresión se deben tener en cuenta las reglas de precedencia y de ser necesario colocar paréntesis para alterarlas.
A continuación se describe el orden en el que serán evaluadas las operaciones:

1. **Not**
2. *** / div mod and**
3. **+ - or**
4. **= <> < <= > >=**

Si dos o mas operadores tienen el mismo nivel de prioridad, se resuelve de izquierda a derecha (asociatividad).

Expresiones lógicas ejemplos

Observación : Una constante y una variable se consideran expresiones (las más elementales)

```
VAR
   a,b,z1,z2,z3,z4,z5,sigue:boolean;
   Alt,peso,zz,c:real;

{Expresiones logicas en asignaciones}
   sigue:= true;
   Z1:= not sigue;
   Z2:= a or  b;
   Z3:=(peso>2) and Z2;
   Z4:=(Alt>=2)or(peso+2=c)and(zz/(5*c)>0);
   Z5:=((Alt>=2)or(peso+2=c))and(zz/(5*c)>0);
```

Expresiones lógicas reglas de precedencia

1. **Not**
2. *** / div mod and**
3. **+ - or**
4. **= <> < <= > >=**

(Alt>=2) or (peso+2=c) and not(zz/5*c >0)

Cuando se evalúa **and**. Su operando derecho es correcto, pero el izquierdo es un valor real (c) ∴ tendremos error! Se deben colocar los parentesis

Se evalúa primero. No hay error porque el operando es una expresión relacional

Expresiones caracter o cadenas

El operador es "+" y sirve para concatenar (unir) caracteres y/o cadenas de caracteres.

Ejemplo:

'h'+'o'+'l'+'a' → 'hola'

'buen'+' '+'día' → 'buen día'